

AI CAREER NAVIGATOR

Using Prompt Engineering

Detailed Project Explanation, Running Code, Examples & Outputs

PROJECT CREATED BY	COURSE
Thallapaka Jagadiswar	GenAI & LLMs
GitHub https://github.com/Jagadiswar2050	LinkedIn https://www.linkedin.com/in/thallapaka-jagadiswar

Project type: Prompt Engineering / Local Career-Advice Demonstration
Implementation: Python + Jupyter Notebook
External API: Not required in the current implementation

Table of Contents

1. Project Overview
2. Objective
3. How the Project Works
4. Prompt Engineering Concepts
5. Complete Running Code
6. Step-by-Step Code Explanation
7. Examples and Expected Outputs
8. How to Run the Project
9. Testing and Validation
10. Architecture
11. Limitations and Future Improvements
12. Viva / Presentation Explanation
13. Conclusion
14. Author Details

1. Project Overview

The project is an **AI Career Navigator using Prompt Engineering**. Its purpose is to demonstrate how a career-advice assistant can be structured using layered prompts and a local Python response engine. The supplied reference material describes a career-advising chatbot based on role prompts, instructions and examples; this report documents the reworked implementation created for **Thallapaka Jagadiswar**.

The current implementation is self-contained. It does not require an external LLM API key. The prompt-engineering layers are assembled in Python, while the response engine uses keyword-based intent routing to produce reproducible career guidance.

Project Components

- **Role Prompt** — establishes CareerNavigator as a practical career mentor.
- **Instruction Prompt** — specifies how responses should be structured.
- **Few-Shot Examples** — demonstrates the desired response style.
- **Prompt Builder** — combines prompt layers with the user question.
- **Intent Router** — detects common career topics.
- **Response Engine** — returns a focused roadmap.
- **Testing** — checks representative career queries.

2. Objective

The objective is to show how prompt engineering can turn a general conversational concept into a focused career-advice workflow. The project concentrates on consistency, structure, simplicity and practical next steps.

- Understand the user's career question.
- Identify an appropriate career track when a known topic is present.
- Provide skills, roadmap items, a project idea and a next step.
- Keep responses concise and easy for students to follow.
- Provide general career guidance when the query does not match a predefined track.

3. How the Project Works

Stage	What happens
1. User Input	Student enters a career question.
2. Prompt Layers	Role, instructions and examples are combined.
3. Query Processing	The question is normalized to lowercase.
4. Intent Detection	Keywords are checked against supported career tracks.
5. Response Generation	A structured roadmap is selected.
6. Output	The roadmap is printed for the user.

Important implementation note: The prompt layers are explicitly constructed for demonstration, but the current response-generation path is a local Python engine rather than a live call to an external large language model.

4. Prompt Engineering Concepts

4.1 Role Prompting

Role prompting defines who the assistant is supposed to behave as. In this project, the role is a professional and practical career mentor.

```
ROLE_PROMPT = """
```

```
You are CareerNavigator, a professional and practical career mentor.
Give concise, structured, beginner-friendly guidance.
Use only information supported by the project knowledge.
Avoid invented facts.
Prefer: Goal -> Skills -> Roadmap -> Project -> Next Step.
"""
```

The role establishes the identity and response philosophy of the assistant.

4.2 Instruction Prompting

```
INSTRUCTION_PROMPT = """
Response rules:
- Understand the user's career intent.
- Identify the most relevant career track.
- Give a short roadmap.
- Recommend practical skills and one project.
- Use bullet points.
- Keep the answer actionable.
"""
```

The instruction prompt describes the expected answer behavior.

4.3 Few-Shot Prompting

```
FEW_SHOT_EXAMPLES = """
Example 1
User: I want to become a data analyst.
Assistant:
Track: Data Analytics
- Learn Python, SQL and spreadsheets
- Practice data cleaning and visualization
- Build a dashboard project

Example 2
User: I want to enter cybersecurity.
Assistant:
Track: Cybersecurity
- Learn networking and Linux basics
- Study security fundamentals
- Practice in legal training environments
"""
```

Few-shot prompting provides examples of the expected input-output pattern.

4.4 Prompt Assembly

```
def build_prompt(user_question: str) -> str:
    return (
        ROLE_PROMPT.strip()
        + "\n\n"
        + INSTRUCTION_PROMPT.strip()
        + "\n\n"
        + FEW_SHOT_EXAMPLES.strip()
        + f"\n\nUser: {user_question}\nAssistant:"
    )
```

The function combines role + rules + examples + the current user query.

5. Complete Running Code

The following is the complete Python implementation included with the project. Save it as **ai_career_navigator.py** and run it with Python.

Code — lines 1 to 60

```

AI Career Navigator using Prompt Engineering
Author: Thallapaka Jagadiswar
Course: GenAI & LLMs
Project Type: Prompt Engineering / Rule-based LLM Simulation

This project demonstrates:
1. Role prompting
2. Instruction prompting
3. Few-shot prompting
4. Prompt assembly
5. Query classification
6. Structured career responses
7. Basic testing
"""

ROLE_PROMPT = """
You are CareerNavigator, a professional and practical career mentor.
Give concise, structured, beginner-friendly guidance.
Use only information supported by the project knowledge.
Avoid invented facts.
Prefer: Goal -> Skills -> Roadmap -> Project -> Next Step.
"""

INSTRUCTION_PROMPT = """
Response rules:
- Understand the user's career intent.
- Identify the most relevant career track.
- Give a short roadmap.
- Recommend practical skills and one project.
- Use bullet points.
- Keep the answer actionable.
"""

FEW_SHOT_EXAMPLES = """
Example 1
User: I want to become a data analyst.
Assistant:
Track: Data Analytics
- Learn Python, SQL and spreadsheets
- Practice data cleaning and visualization
- Build a dashboard project
- Publish the project with documentation

Example 2
User: I want to enter cybersecurity.
Assistant:
Track: Cybersecurity
- Learn networking and Linux basics
- Study security fundamentals
- Practice in legal training environments
- Document small security-learning projects
"""

def build_prompt(user_question: str) -> str:
    """Assemble the layered prompt used by the project."""
    return (
        ROLE_PROMPT.strip()
        + "\n\n"
        + INSTRUCTION_PROMPT.strip()

```

Code — lines 61 to 120

```

        + "\n\n"
        + FEW_SHOT_EXAMPLES.strip()
        + f"\n\nUser: {user_question}\nAssistant:"
    )

def career_response(user_text: str) -> str:
    """
    Lightweight local response engine used for demonstration.
    It simulates the behavior expected from the layered prompt.
    """
    user = user_text.lower().strip()

    if any(k in user for k in ["data analyst", "data analysis", "analytics"]):
        return """Track: Data Analytics
- Learn Python, SQL and spreadsheet fundamentals
- Practice data cleaning and visualization
- Build a sales/college-data dashboard

```

```

- Publish the project on GitHub
Next step: complete one small dataset project this week."""

    if any(k in user for k in ["ai", "machine learning", "ml", "artificial intelligence"]):
        return """Track: AI / Machine Learning
- Strengthen Python, NumPy and Pandas
- Learn regression, classification and clustering
- Practice with scikit-learn
- Build a prediction or classification project
Next step: choose one dataset and complete an end-to-end mini project."""

    if any(k in user for k in ["cyber", "security", "ethical hacking"]):
        return """Track: Cybersecurity
- Learn networking and Linux fundamentals
- Study security concepts and safe testing practices
- Practice in authorized training labs
- Build a small security-awareness or log-analysis project
Next step: create a weekly networking + Linux practice plan."""

    if any(k in user for k in ["web", "frontend", "backend", "developer", "software"]):
        return """Track: Software Development
- Learn one programming language deeply
- Practice Git, problem solving and basic databases
- Build small web applications
- Document projects with README files
Next step: build one useful CRUD-style application."""

    if any(k in user for k in ["resume", "cv", "portfolio"]):
        return """Track: Resume & Portfolio
- Keep the resume concise and project-focused
- Highlight technical skills with evidence
- Add 1-2 meaningful projects with links
- Use action verbs such as Built, Designed and Implemented
Next step: update one project description today."""

    if any(k in user for k in ["ece", "mechanical", "commerce", "non it", "switch to it"]):
        return """Track: Transition to IT
- Start with Python and SQL fundamentals
- Choose one entry path: Development, Data or Testing
- Build 1-2 beginner projects
- Create a clear portfolio and resume
Next step: choose one track and follow it for 30 days."""

```

Code — lines 121 to 161

```

    return """Track: General Career Planning
- Identify one target role
- Learn the core skills for that role
- Build a small practical project
- Improve your resume and communication
- Apply for internships or trainee opportunities
Next step: tell CareerNavigator your target role for a more focused roadmap."""

def career_navigator(user_question: str) -> str:
    """Public function used by the notebook and command line."""
    _prompt = build_prompt(user_question)
    return career_response(user_question)

def run_tests():
    """Simple functional tests for the project."""
    cases = {
        "I want to become a data analyst": "Data Analytics",
        "How can I start machine learning?": "AI / Machine Learning",
        "I want to learn cybersecurity": "Cybersecurity",
        "How do I improve my resume?": "Resume & Portfolio",
        "I want to become a software developer": "Software Development",
    }
    results = []
    for query, expected in cases.items():
        actual = career_navigator(query)
        passed = expected in actual
        results.append((query, expected, passed))
    return results

if __name__ == "__main__":
    print("=" * 60)
    print("AI CAREER NAVIGATOR — THALLAPAKA JAGADISWAR")
    print("=" * 60)
    print("Type 'exit' to stop.\n")

    while True:
        question = input("Career question: ").strip()
        if question.lower() == "exit":
            print("Thank you for using CareerNavigator.")
            break
        print("\n" + career_navigator(question) + "\n")

```

6. Step-by-Step Code Explanation

6.1 Input normalization

```
user = user_text.lower().strip()
```

Converts the question to lowercase and removes surrounding spaces so keyword checks are consistent.

6.2 AI / ML detection

```
if any(k in user for k in ["ai", "machine learning", "ml"]):
```

Routes AI/ML questions to the AI / Machine Learning roadmap.

6.3 Data detection

```
if any(k in user for k in ["data analyst", "data analysis", "analytics"]):
```

Routes data-related questions to the Data Analytics roadmap.

6.4 Cybersecurity detection

```
if any(k in user for k in ["cyber", "security", "ethical hacking"]):
```

Routes cybersecurity questions to the security roadmap.

6.5 Software detection

```
if any(k in user for k in ["web", "frontend", "backend", "developer", "software"]):
```

Routes software-development questions to the development roadmap.

6.6 Resume detection

```
if any(k in user for k in ["resume", "cv", "portfolio"]):
```

Routes resume and portfolio questions to career-document guidance.

6.7 Fallback

```
return """Track: General Career Planning ..."""
```

Provides general career advice when no supported topic is detected.

7. Examples and Expected Outputs

Example 1 — AI / Machine Learning

Input: I want to learn AI

Expected output:

Track: AI / Machine Learning

- Strengthen Python, NumPy and Pandas
- Learn regression, classification and clustering
- Practice with scikit-learn
- Build a prediction or classification project

Next step: choose one dataset and complete an end-to-end mini project.

Example 2 — Data Analytics

Input: How do I become a data analyst?

Expected output:

Track: Data Analytics

- Learn Python, SQL and spreadsheet fundamentals
- Practice data cleaning and visualization
- Build a sales/college-data dashboard
- Publish the project on GitHub

Next step: complete one small dataset project this week.

Example 3 — Cybersecurity

Input: How can I start cybersecurity?

Expected output:

Track: Cybersecurity

- Learn networking and Linux fundamentals
- Study security concepts and safe testing practices
- Practice in authorized training labs
- Build a small security-awareness or log-analysis project

Next step: create a weekly networking + Linux practice plan.

Example 4 — Resume

Input: How can I improve my resume?

Expected output:

Track: Resume & Portfolio

- Keep the resume concise and project-focused
- Highlight technical skills with evidence
- Add 1-2 meaningful projects with links
- Use action verbs such as Built, Designed and Implemented

Next step: update one project description today.

Example 5 — Fallback

Input: I am confused about my career

Expected output:

Track: General Career Planning

- Identify one target role
- Learn the core skills for that role
- Build a small practical project
- Improve your resume and communication
- Apply for internships or trainee opportunities

Next step: tell CareerNavigator your target role for a more focused roadmap.

8. How to Run the Project

8.1 Run as a Python file

- Install Python 3.x.
- Open the project folder in VS Code or a terminal.
- Run the command below.
- Enter a career question when prompted.
- Type **exit** to stop.

```
python ai_career_navigator.py
```

8.2 Run in Jupyter Notebook

- Open **AI_Career_Navigator_Jagadiswar.ipynb** in Jupyter, JupyterLab or VS Code.
- Run the prompt-design cells.
- Run the response-engine cell.
- Run the demonstration/test cell.
- Change the test queries to experiment with the system.

8.3 Example terminal session

```
AI CAREER NAVIGATOR — THALLAPAKA JAGADISWAR
Type 'exit' to stop.
```

```
Career question: I want to learn AI
```

```
Track: AI / Machine Learning
- Strengthen Python, NumPy and Pandas
- Learn regression, classification and clustering
- Practice with scikit-learn
- Build a prediction or classification project
Next step: choose one dataset and complete an end-to-end mini project.
```

```
Career question: exit
Thank you for using CareerNavigator.
```

9. Testing and Validation

The project contains a small test function that sends representative questions through the public interface.

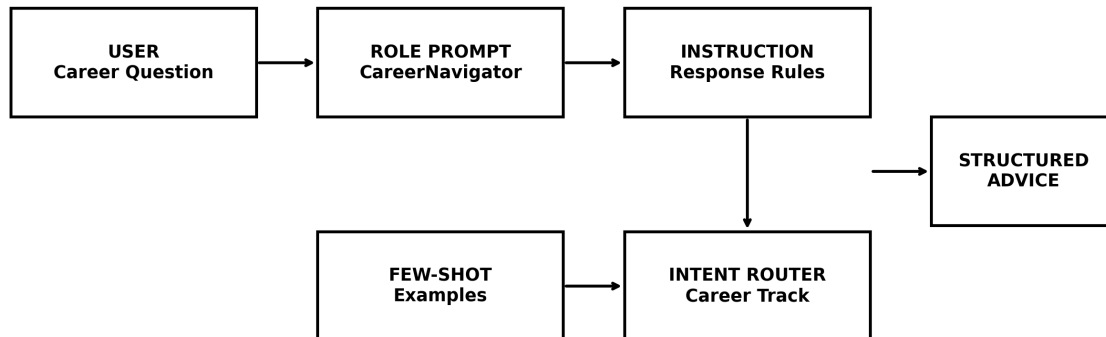
```
def run_tests():
    cases = {
        "I want to become a data analyst": "Data Analytics",
        "How can I start machine learning?": "AI / Machine Learning",
        "I want to learn cybersecurity": "Cybersecurity",
        "How do I improve my resume?": "Resume & Portfolio",
        "I want to become a software developer": "Software Development",
    }
    results = []
    for query, expected in cases.items():
        actual = career_navigator(query)
        passed = expected in actual
        results.append((query, expected, passed))
    return results
```

Test query	Expected track	Purpose
I want to become a data analyst	Data Analytics	Topic routing
How can I start machine learning?	AI / Machine Learning	AI/ML routing
I want to learn cybersecurity	Cybersecurity	Security routing
How do I improve my resume?	Resume & Portfolio	Resume routing
I want to become a software developer	Software Development	Developer routing

This is functional testing: it checks that representative queries are routed to the intended category. It is not a statistical evaluation of an LLM.

10. Architecture

AI Career Navigator — Prompt Engineering Architecture



User → Role Prompt → Instruction Prompt → Few-Shot Examples → Intent Router → Structured Career Advice

The architecture separates prompt design from response routing. A future implementation can replace the local response engine with a real LLM API while retaining the prompt layers.

11. Limitations and Future Improvements

Current limitations

- The response engine is keyword/rule based.
- It does not call an external LLM API.
- Career knowledge is manually defined in the source code.
- There is no persistent user profile or conversation history.
- There is no live job, course or labor-market data connection.

Future improvements

- Connect the prompt builder to a real LLM API.
- Add student profiles and skill-gap analysis.
- Add a curated career knowledge base.
- Add resume analysis and personalized roadmaps.
- Create a web interface.
- Add secure progress tracking.

12. Viva / Presentation Explanation

30-second explanation

My project is **AI Career Navigator using Prompt Engineering**. The objective is to build a career-advice assistant that gives structured guidance to students. I use three main prompt-engineering techniques: role prompting, instruction prompting and few-shot prompting. The role prompt defines the assistant as a professional career mentor, the instruction prompt controls the response format, and few-shot examples demonstrate the expected answer style. The Python implementation receives a user's career question, identifies the relevant career area, and generates a structured roadmap containing skills, projects and next steps. The current implementation uses a local response engine, so it works without an external API. It can later be connected to a real LLM.

Q: What is prompt engineering?

A: Designing and structuring instructions/examples to guide the behavior and format of an AI system.

Q: Why use a role prompt?

A: To define the assistant's intended persona and response behavior.

Q: What is few-shot prompting?

A: Providing example input-output pairs to demonstrate the desired response pattern.

Q: Does the current project call an LLM API?

A: No. It demonstrates the prompt structure and uses a local Python response engine.

Q: Why use a local engine?

A: It keeps the project reproducible and avoids API keys and network dependency.

Q: Can this become a real AI chatbot?

A: Yes. The prompt builder can later be connected to an LLM API.

Q: What happens for an unknown question?

A: The fallback route provides general career-planning guidance.

13. Conclusion

AI Career Navigator demonstrates a complete small-scale prompt-engineering workflow. It shows how a role definition, response rules and few-shot examples can be organized into a reusable prompt structure, while Python provides a reproducible local demonstration layer.

The implementation is transparent: the present version does not claim to be a live LLM integration. It provides a clean foundation that can later be connected to an actual language model service.

14. Author Details

Name	Thallapaka Jagadiswar
Project	AI Career Navigator using Prompt Engineering
Course	GenAI & LLMs
GitHub	https://github.com/Jagadiswar2050
LinkedIn	https://www.linkedin.com/in/thallapaka-jagadiswar

Project Demonstration Preview

AI Career Navigator — Demonstration

```
USER
I want to learn AI and build projects.

CAREERNAVIGATOR RESPONSE
Track: AI / Machine Learning
• Strengthen Python, NumPy and Pandas
• Learn regression, classification and clustering
• Practice with scikit-learn
• Build a prediction or classification project
Next step: complete one end-to-end mini project.
```